

EXAMEN PRACTICO: TESTING DE CAJA NEGRA

Caso: Biblioteca Municipal "Lectura Viva"

Tecnicas evaluadas: Particiones de Equivalencia y Analisis de Valores Limite (AVL)

1) Contexto del examen

La biblioteca necesita validar reglas de negocio en su software. Se te proporciona la clase Biblioteca con cuatro metodos. Tu tarea es disenar y escribir tests de caja negra con JUnit 5, sin depender de la implementacion interna.

2) Enunciado funcional (especificacion)

Metodo 1: calcularCuotaCarnet(int edad, boolean familiaNumerosa)

- Si edad < 0 -> IllegalArgumentException
- Si edad < 14 -> cuota = 8.0 EUR
- Si 14 <= edad <= 64 -> cuota = 25.0 EUR
- Si edad >= 65 -> cuota = 12.0 EUR
- Si familiaNumerosa = true -> descuento del 20% sobre la cuota resultante

Metodo 2: clasificarLecturaMensual(int librosLeidos)

- Si librosLeidos < 0 -> IllegalArgumentException
- Si librosLeidos < 2 -> "Ocasional"
- Si 2 <= librosLeidos < 5 -> "Frecuente"
- Si librosLeidos >= 5 -> "Intensiva"

Metodo 3: calcularPenalizacionRetraso(String tipoMaterial, int diasRetraso)

- Si diasRetraso < 0 -> IllegalArgumentException
- Si tipoMaterial es desconocido o null -> IllegalArgumentException
- Tarifas por dia de retraso:
 - - libro: 0.50 EUR/dia
 - - revista: 0.25 EUR/dia
 - - dvd: 1.50 EUR/dia
- Resultado: penalizacion = tarifaDia * diasRetraso

Metodo 4: validarCodigoPrestamo(String codigo)

- Si codigo == null -> false
- Si longitud != 10 -> false
- Formato valido: LL-DDDDDDL (2 letras, guion, 6 digitos, 1 letra)
- Ejemplo valido: AB-123456Z

3) Que se espera en tu solucion de tests

- Un archivo de test (por ejemplo BibliotecaTest.java) con anotaciones de JUnit 5.
- Analisis visible por metodo: particiones validas e invalidas.
- Pruebas de valores limite en cada frontera relevante.
- Casos de excepcion con assertThrows.
- Casos parametrizados con @ParameterizedTest y @CsvSource donde tenga sentido.
- Nombres claros de test con @DisplayName.

4) Criterios de correccion (orientativos)

Bloque	Peso
Metodo 1: calcularCuotaCarnet	2.5 puntos
Metodo 2: clasificarLecturaMensual	2.5 puntos
Metodo 3: calcularPenalizacionRetraso	2.5 puntos
Metodo 4: validarCodigoPrestamo	2.5 puntos
Total	10 puntos

5) Pistas tecnicas de examen

- Empieza por casos invalidos: suelen ser los mas claros del contrato.
- En limites, prueba siempre limite-1, limite exacto y limite+1.
- Para doubles, usa assertEquals(esperado, real, delta).
- No mezcles imports de JUnit 4; usa org.junit.jupiter.*.
- Estructura tus tests con patron AAA: Arrange, Act, Assert.

6) Entrega recomendada

Entrega un fichero BibliotecaTest.java limpio y ejecutable, con cobertura de reglas del enunciado. Si adjuntas comentarios de particiones y limites por metodo, mejorara la legibilidad y la correccion.

Fin del enunciado